

Project Report: Mr.HelpMate AI — Generative Search System for Insurance Policy

1. Project Overview

The objective of this project was to design and implement a **generative search system** for a group life insurance policy document. Named **Mr.HelpMate AI**, the system allows users to input questions about the policy and receive relevant, context-aware answers derived directly from the policy content.

The motivation behind this project is practical: insurance documents are often long and complex, making it challenging for members or employees to quickly find precise information. By building a search system powered by embeddings, reranking, and a generation layer, the goal was to make information retrieval faster, accurate, and more user-friendly.

2. Data and Preprocessing

The dataset consists of a single **Group Member Life Insurance Policy PDF**. The following steps were carried out:

1. **Extraction:** Using **PyPDF2**, text from all pages of the PDF was extracted.
2. **Cleaning:** The text was cleaned to remove extra spaces, unnecessary line breaks, and carriage returns.
3. **Chunking:** The cleaned text was split into overlapping chunks of 800 characters with an overlap of 200.
 - This ensures context is preserved across chunks and improves search quality.
 - Total chunks indexed: **173**.

A **chunk length histogram** was generated to visualize the distribution of chunk sizes and verify the chunking process.

3. Embedding Layer

Each chunk was converted into a vector representation using the **SentenceTransformers** model **all-MiniLM-L6-v2**.

- Purpose: capture semantic meaning of each chunk so that similar queries and chunks can be matched effectively.
- Embeddings were stored in **ChromaDB**, allowing efficient vector-based retrieval.

This layer forms the foundation of the search system, enabling semantic rather than keyword-based search.

4. Search Layer

The search layer retrieves candidate chunks based on the semantic similarity between the query and chunk embeddings.

Key design points:

- **Query Embeddings:** Each user question is converted into a vector using the same SentenceTransformer model.
- **Top-K Retrieval:** The system initially retrieves the top 10 closest chunks from ChromaDB.
- **Reranking:** Using a **Cross-Encoder model (cross-encoder/ms-marco-MiniLM-L-6-v2)**, the top chunks are reranked to improve accuracy.

For evaluation, 3 self-designed queries were tested:

1. Eligibility criteria for group member life insurance
2. Death benefit for accidental death under the policy
3. Procedure for filing a claim and required documentation

For each query, the **top-3 chunks** were printed and used in the generation layer.

5. Generation Layer

The generation layer produces a coherent answer based on the top-ranked chunks from the search layer.

- For simplicity, the system currently outputs the **first snippet** from the top-3 chunks with a small context preview.
- This was sufficient to demonstrate the retrieval-augmented generation workflow.
- Example: For the eligibility query, the system returned the exact clause describing the 30-day active work requirement for policy members.

Optional **similarity bar charts** were also generated to visualize the top-5 chunk relevance per query.

6. Implementation Highlights

- **Pipeline Structure:** Embedding → Search → Reranking → Generation
- **Caching:** Query results are cached to speed up repeated searches.
- **Visualization:** Added top-chunk similarity charts and chunk length histogram to understand model behavior.
- **Reproducibility:** Fully implemented in Google Colab without external API keys.

7. Challenges Faced

1. **Chunking strategy:** Initially considered sentence-based chunking but opted for character-based chunks due to simplicity and predictable chunk sizes.
2. **Reranking performance:** Cross-Encoder reranking is computationally heavier; careful selection of top-k candidates was necessary to balance speed and accuracy.
3. **Answer generation:** Keeping the answer concise while providing context required limiting snippet length.

8. Lessons Learned

- Embedding-based semantic search is powerful for domain-specific documents.
- Reranking improves retrieval accuracy significantly compared to raw embeddings.
- Visualization is extremely helpful for understanding retrieval behavior and debugging.
- Even with a single document, careful chunking and embedding choices greatly impact result quality.
- Building a fully functional retrieval-augmented generation system does not necessarily require large-scale datasets—structure and pipeline design matter most.

9. Conclusion

Mr.HelpMate AI demonstrates a complete **retrieval-augmented generative search pipeline** for an insurance policy document.

- Users can now ask questions in natural language and get precise, contextually accurate answers.
- The project highlights the importance of **chunking strategy, embedding model selection, reranking, and generation layer design**.
- Optional visualizations provide transparency into system behavior, making it easier to debug and improve.

Overall, this project strengthens skills in **document processing, embeddings, vector databases, reranking, and retrieval-augmented generation**, with direct practical applications in the insurance domain.